End-to-End Argo CD GitOps Deployment Guide

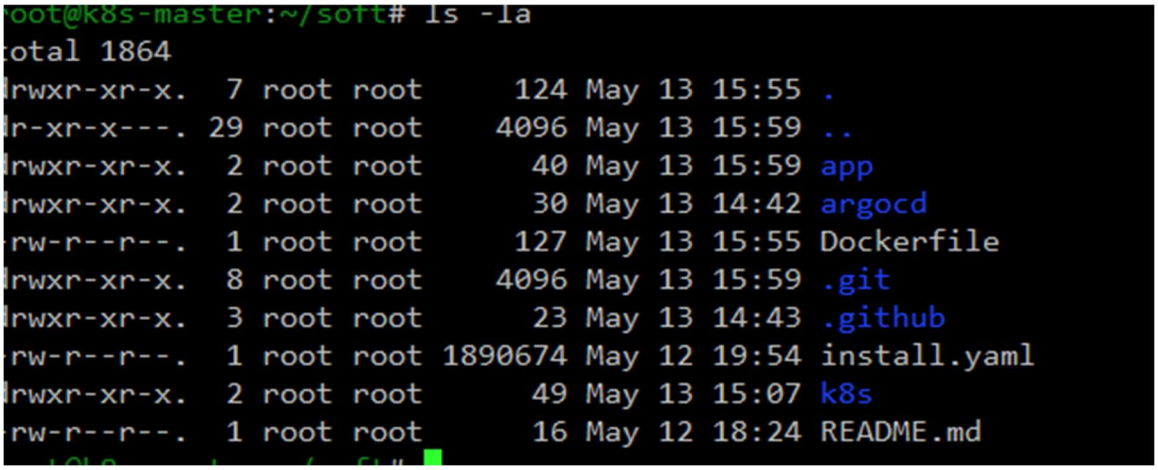
1. Overview

This document provides a step-by-step guide to building a small application, containerizing it, deploying it to Kubernetes, and managing deployment using Argo CD (GitOps approach).

2. Application Code

Mkdir soft

Mkdir app



cd app

app.js:

```
const express = require("express");
const app = express();
app.get("/", (req, res) => {
  res.send("Hello from ArgoCD deployed app 🚀 ");
});
app.listen(3000, () => console.log("Running on port 3000"));
```

package.json:

```
{
  "name": "argocd-demo",
  "version": "1.0.0",
  "dependencies": {
```

```
   "express": "^4.18.2"
 }
}
```

3. Docker Configuration

Dockerfile:

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Build & Push:

```
docker build -t kaisarkhan1981/argocd-demo:v1 .
docker push kaisarkhan1981/argocd-demo:v1
```

4. Kubernetes Manifests

```
mdkir k8s
```

deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: argocd-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: argocd-demo
  template:
    metadata:
      labels:
        app: argocd-demo
    spec:
      containers:
      - name: argocd-demo
        image: kaisarkhan1981/argocd-demo:v1
```

```
      ports:
      - containerPort: 3000
```

service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: argocd-demo-service
spec:
  type: NodePort
  selector:
    app: argocd-demo
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30007
```

* Create projects on GitHub and push files from local to GitHub

5. Install Argo CD

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

6. Access Argo CD

```
kubectl port-forward svc/argocd-server -n argocd 8080:443
kubectl get secret argocd-initial-admin-secret -n argocd -o jsonpath="{.data.password}" | base64 --decode
```

Step 7: Login

Username: admin

Password: from previous step

Step 8: Install Argo CD CLI

```
curl -sSL -o argocd \
```

https://github.com/argoproj/argocd/releases/latest/download/argocd-linux-amd64

chmod +x argocd

#sudo mv argocd /usr/local/bin/

Step 9: CLI Login

argocd login localhost:8080

argocd app sync argocd-demo

10. Argo CD Application

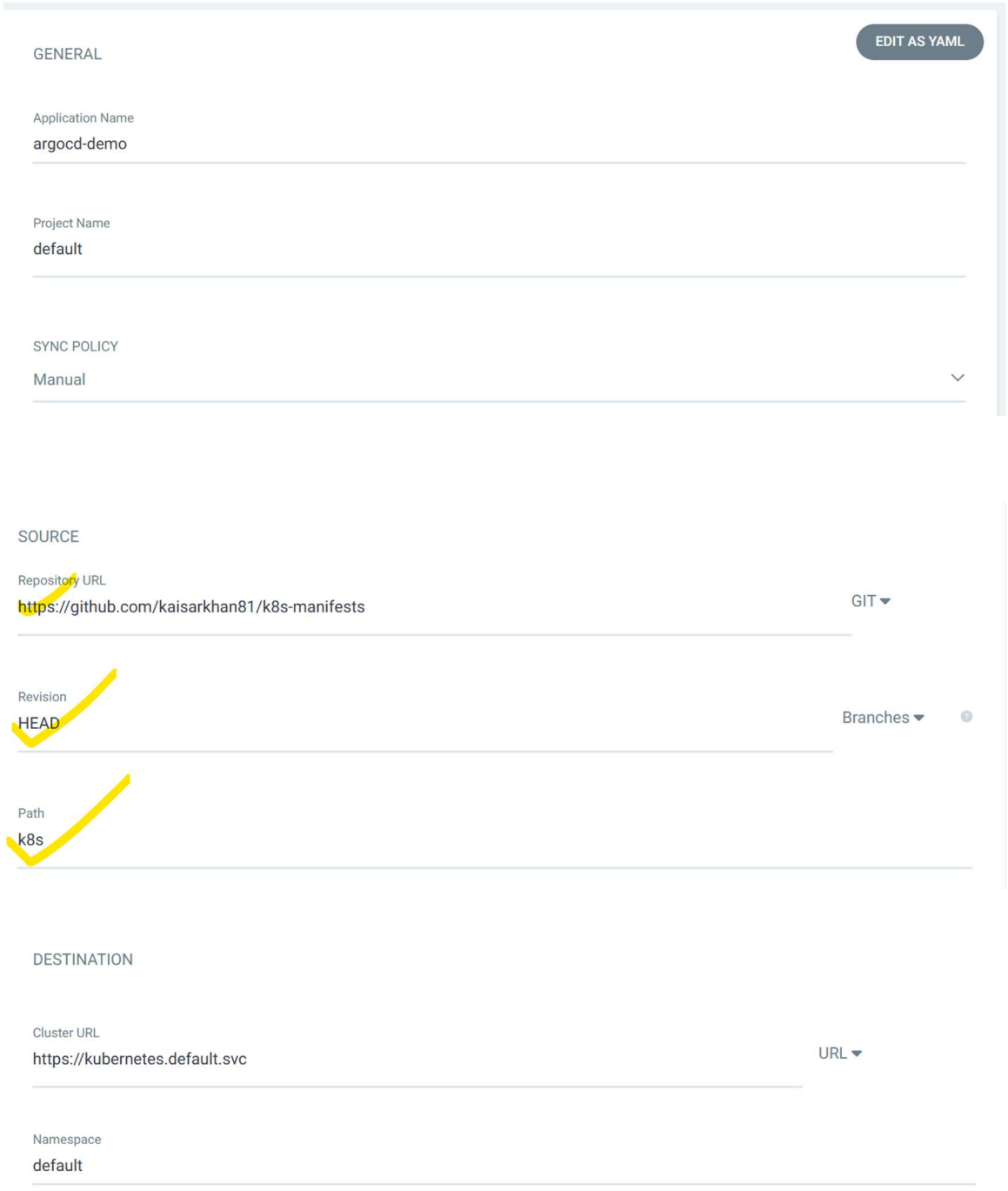
mkdir argocd

application.yaml:

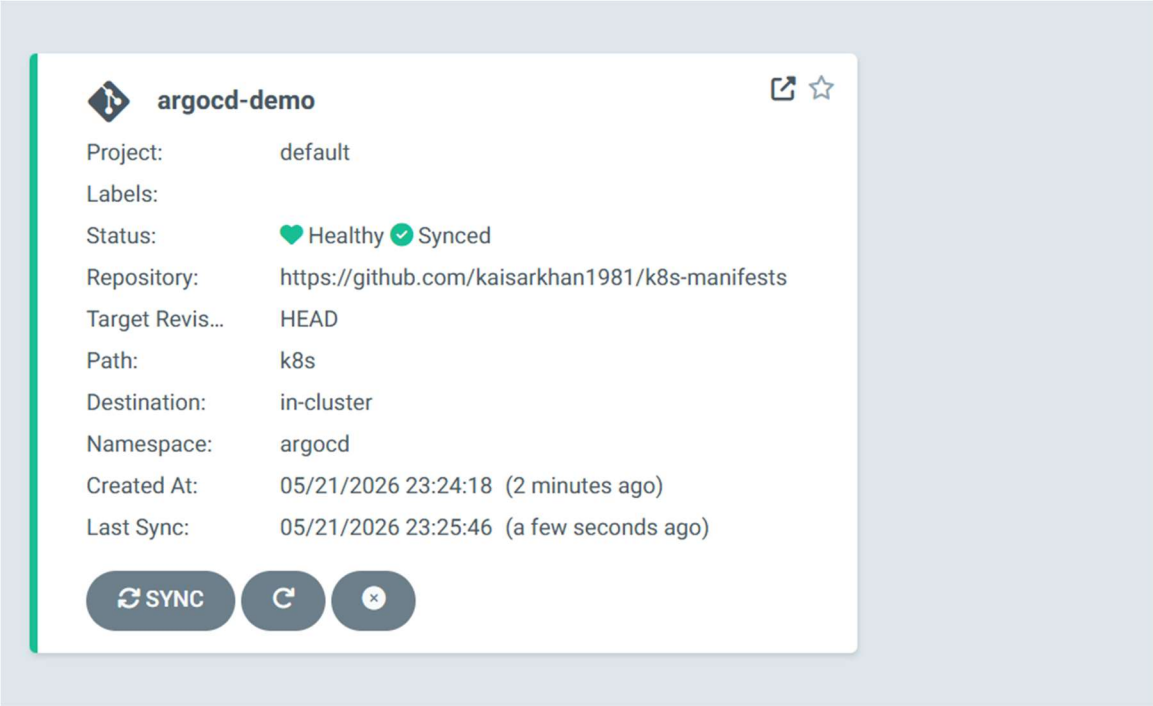
```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: argocd-demo
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/kaisarkhan81/k8s-manifests
    targetRevision: HEAD
    path: k8s
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

or Graphically

Applications > new Applications >



Rest of the things remain default



11. GitHub Actions CI/CD

mkdir –p .github/workflows

.github/workflows/docker-build.yml

```
name: Build and Push Docker Image
on: [push]
jobs:
 build:
   runs-on: ubuntu-latest
   steps:
   - uses: actions/checkout@v4
   - uses: docker/setup-buildx-action@v3
   - uses: docker/login-action@v3
     with:
       username: ${{ secrets.DOCKER_USERNAME }}
       password: ${{ secrets.DOCKER_PASSWORD }}
   - run: |
       docker build -t kaisarkhan1981/argocd-demo:${{ github.sha }} .
       docker push kaisarkhan1981/argocd-demo:${{ github.sha }}
```

12. Create secret on github for docker user and tokens

Personal Access Tokens (Classic) for github login

Personal access tokens | Docker

13. Add secret docker_username and docker_password

1. Go to your **GitHub repository**
2. Click on **Settings** (top menu)
3. In the left sidebar:
    - Go to **Security & Quality**
    - Click **Secrets and variables**
    - Select **Actions**
4. Click **New repository secret**

➕ **Add First Secret (Docker Username)**

- **Name:** docker_username
- **Secret value:** Your Docker Hub username
- Click **Add secret**

➕ **Add Second Secret (Docker Password)**

- Click **New repository secret** again
- **Name:** docker_password
- **Secret value:** Your Docker Hub password (or better: **Access Token**)
- Click **Add secret**

✅ **Best Practice (Recommended)**

Instead of using your real password:

- Go to Docker Hub → **Account Settings → Security**

- Create an **Access Token**
- Use that token as docker_password

12. Verification
kubectl get pods
kubectl get svc
Access app via http://<node-ip>:30007

13. Update service IP manually

kubectl patch svc agrocd-server -n argocd -p '{"spec":{"externalIPs":["192.168.0.106"]}}'

iIrmZfHwlL1pZbKC